BME INSTITUTE - TALLER B1-T1

Multi-Source Portfolio Analytics and Monte Carlo Toolkit

Proposed implementation summary for a three-person team

Project intent

Build a robust, modular Python toolbox that satisfies every compulsory requirement of the workshop while adding a small number of quantitatively meaningful extensions: a realistic multi-asset portfolio use case, correlated Gaussian and Student-t Monte Carlo, richer risk analytics, and an optional option-implied-volatility module.

Workshop deadline: 12 November 2026, 18:00
Deliverable: GitHub repository + 5-minute presentation

Prepared for team discussion - 6 October 2026

Contents

1. Executive summary
2. Workshop requirements and scope mapping
3. Proposed use case and portfolio
4. Target software architecture
5. Market-data provider strategy
6. Domain models and data standardisation
7. Ingestion, cleaning and preprocessing
8. Analytics layer
9. Monte Carlo engine
10. Optional options / implied-volatility extension
11. Package and repository structure
12. Reporting and visualisation
13. Testing, reproducibility and engineering standards
14. Three-person work split
15. Milestones to 12 November
16. Risks and scope control
17. Definition of done and presentation storyline

Appendix A. Suggested interfaces and equations

Appendix B. Source notes

1. Executive summary

The workshop brief asks for a Python toolbox that obtains and standardises stock-market information from multiple online APIs, models price series and a portfolio through DataClasses, calculates automatic statistics, performs Monte Carlo simulation for both single assets and the overall portfolio, and produces analytical reports and plots. The pedagogical emphasis is explicitly on sound software design: abstractions, modularity, decoupling, interoperable internal formats and a plug-and-play repository.

Our proposed implementation deliberately keeps that compulsory core simple and reliable, then adds extensions behind interchangeable interfaces. This avoids over-engineering the assignment while still producing a project that looks like a compact institutional portfolio-risk engine rather than a collection of disconnected scripts.

Recommended scope hierarchy

Core first: provider abstraction, standardised DataClasses, multi-series extraction, cleaning, portfolio model, Gaussian correlated Monte Carlo, .report(), .plots_report(), README/tests/UML. Differentiation second: Student-t Monte Carlo, VaR/Expected Shortfall, richer statistics and a realistic 70/30 multi-asset example. Advanced bonus last: options, implied-volatility calibration and possibly local volatility.

2. Workshop requirements and scope mapping

The following mapping keeps the team aligned with what is actually graded. Everything marked "extension" should be implemented only after the corresponding compulsory feature is stable.

Workshop requirement	Proposed implementation	Scope
GitHub + /src + plug-and-play	pyproject.toml or requirements.txt, .gitignore, README, example commands, reproducible environment	CORE
Multi-source ingestion	Common MarketDataProvider interface with several adapters	CORE
Stocks and indices	Yahoo/yfinance + Alpha Vantage adapters; batch retrieval	CORE
Additional data type	FRED and/or ECB rates/macro series	CORE
Standardised output	Provider responses mapped into internal DataClasses	CORE
Price-series DataClass	Metadata, prices, returns and automatic mean/std	CORE
Portfolio DataClass	Assets, weights, validation, aggregate returns and risk	CORE
Monte Carlo	Configurable paths, horizon, return and volatility; asset + portfolio	CORE
Visualisation	Simulation paths, terminal distribution, performance/correlation views	CORE
Cleaning/preprocessing	NaNs, calendars, alignment, duplicate dates, frequency rules	CORE
.report() Markdown	Metrics, diagnostics and warnings	CORE
.plots_report()	Analytical plots collected behind one	CORE

	method	
Formal architecture diagram	UML / dependency-flow view	CORE
Student-t / VaR / ES	Alternative fat-tail simulation and risk comparison	EXTENSION
Options / IV surface	MarketData.app/yfinance + own IV solver; local vol only if time permits	ADVANCED

3. Proposed use case and portfolio

3.1 Project question

A useful framing is: "How does a diversified, AI-tilted multi-asset portfolio behave under historical and stochastic future scenarios, and how sensitive are the conclusions to Gaussian versus fat-tailed return assumptions?" This gives the toolbox a coherent financial use case without restricting the library to one hard-coded portfolio.

3.2 Example portfolio for demonstration

Sleeve	Illustrative weight	Possible instrument/factor	Why it is useful
US broad equity	30%	S&P 500 (index or ETF proxy)	Core equity beta
US growth	15%	Nasdaq-100	Growth / technology concentration
Semiconductors	10%	SOX or semiconductor ETF proxy	AI hardware cycle
Single-name AI infrastructure	7.5%	ASML	Non-US semiconductor supply chain
Single-name foundry	7.5%	TSMC / ADR	Major AI manufacturing exposure
US sovereign fixed income	15%	Treasury ETF or rate-driven proxy	Rates diversification
EUR sovereign fixed income	15%	EUR government-bond proxy	EUR rates diversification

The weights are not a mandatory design choice; they are a demonstration scenario. The library should accept arbitrary tickers, series and weights, subject to explicit validation rules. A simpler portfolio can be used in automated tests.

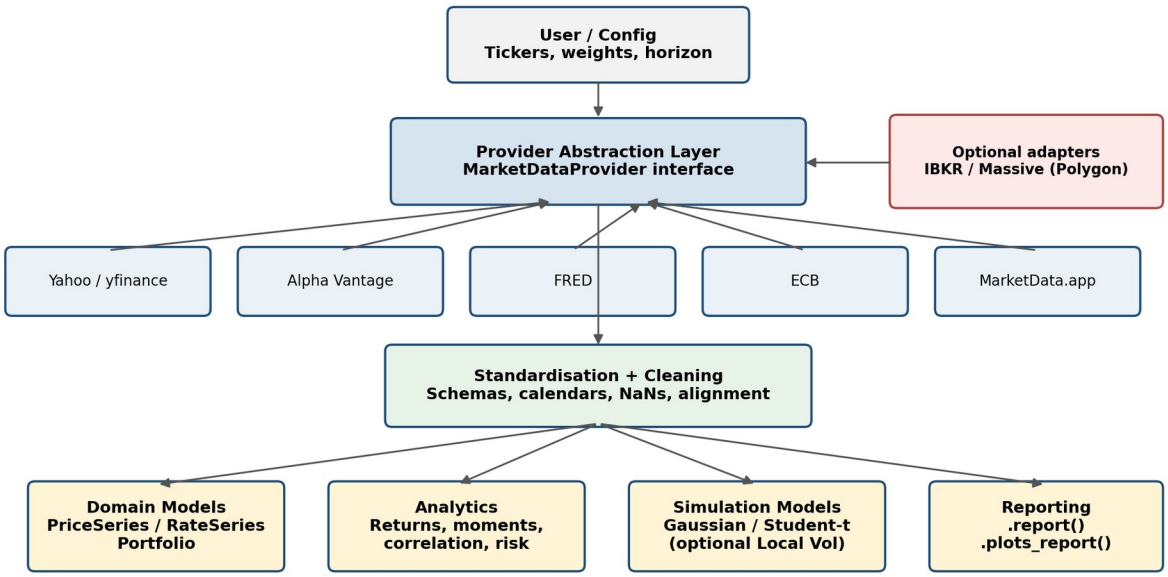
3.3 Risk factors we may model

- **Equity prices:** spot/adjusted-close histories for indices, ETFs and single names.
- **Interest rates:** US and EUR rate series from FRED/ECB; initially as portfolio inputs/benchmarks, with rates-driven bond valuation as an optional enhancement.
- **FX:** EUR/USD becomes relevant if the portfolio base currency is EUR and some assets are USD-denominated.
- **Volatility:** historical realised volatility in the core model; option-implied volatility as an optional market-implied extension.

- **Dependence:** covariance/correlation is essential because portfolio simulation must not treat assets independently.

4. Target software architecture

The architecture should make the ingestion layer replaceable. Analytics and simulation code should consume internal domain objects, never raw Yahoo, Alpha Vantage, FRED, ECB or option-provider payloads.



Core is provider-independent: adding or removing one API should not alter portfolio or simulation logic.

4.1 Design principles

- **Provider independence:** a provider adapter can fail or be removed without changing portfolio or simulation logic.
- **Common schemas:** equivalent data from different APIs must map to the same internal representation.
- **Single responsibility:** data retrieval, cleaning, analytics, simulation and reporting live in separate modules.
- **Extensibility:** new providers and simulation models are added by implementing an interface rather than editing a monolithic script.
- **Reproducibility:** configuration, seeds, date ranges and dependencies must be explicit.
- **Graceful degradation:** advanced modules such as options should be optional; the core project must run without paid credentials.

5. Market-data provider strategy

The brief names Yahoo/yfinance, Alpha Vantage, Finnhub and Polygon as examples. We propose a smaller, purpose-driven set of adapters. Four well-tested providers are preferable to many superficial

integrations. Provider limits and licences can change, so all credentials and entitlements should be treated as configuration rather than assumptions embedded in code.

Provider	Reachable from Python?	Primary role	Important caveat
Yahoo / yfinance	Yes - third-party Python library	Equity/index/ETF history; dividends; optional option-chain cross-check	Convenient but unofficial; API behaviour may change
Alpha Vantage	Yes - official REST API + key	Second market-data source; equities, FX, fundamentals/macros as needed	Rate limits and some endpoints are premium
FRED	Yes - official REST API + key	US rates and macroeconomic time series	Economic series have different frequencies/calendars
ECB Data Portal	Yes - official SDMX REST API	EUR rates, FX and euro-area statistics	SDMX dimensions require careful query construction
MarketData.app	Yes - official REST API + token	Primary optional source for option chains, IV and Greeks	Free tier is delayed and limited; full chains can consume many credits
Interactive Brokers	Yes - official TWS/IB Gateway API	Optional advanced/live-market adapter	Operationally heavier; live subscriptions require entitlements
Massive / Polygon	Yes - official API	Optional additional market/options adapter	Useful as a fallback or extra adapter; not necessary for core

Recommended implementation order

Core adapters: Yahoo/yfinance + Alpha Vantage + FRED + ECB. Option extension: MarketData.app first; yfinance option chain as an independent cross-check. IBKR and Massive/Polygon are optional adapters and must not be prerequisites for cloning and running the repository.

5.1 Current feasibility notes (checked 6 October 2026)

- **yfinance:** documented Python calls support Ticker.history(), option_chain() and multi-ticker download.
- **Alpha Vantage:** official documentation exposes stock/index/options endpoints through API keys; premium status varies by endpoint.
- **FRED:** official API returns observations programmatically and supports JSON/CSV/XML workflows.
- **ECB:** official SDMX 2.1 REST service provides programmatic statistical-data retrieval and multiple output formats.
- **MarketData.app:** option-chain endpoint exposes contract quotes plus IV/Greeks; the current free plan is suitable for prototyping with delayed data and limited daily credits.
- **IBKR:** free delayed data exists for many instruments, while live data requires subscriptions; the USD 500 minimum relates to maintaining paid market-data/research subscriptions, not to the mere existence of the API.

6. Domain models and data standardisation

The most important object-design decision is the internal schema. Provider payloads should be converted immediately into domain objects. This is directly aligned with the workshop requirement that all extractors return homogeneous, interoperable objects.

6.1 Core DataClasses

```
@dataclass
class PriceSeries:
    symbol: str
    source: str
    asset_class: str
    currency: str
    frequency: str
    prices: pd.Series
    metadata: dict
    mean_return: float = field(init=False)
    volatility: float = field(init=False)

@dataclass
class Portfolio:
    positions: list[Position]
    base_currency: str = "EUR"
    # methods: returns(), statistics(), simulate(), report(), plots_report()
```

- **PriceSeries:** canonical date index, values/prices, source metadata, symbol, currency, asset class and automatic statistics.
- **Position:** links an asset/series to a portfolio weight or quantity.
- **Portfolio:** validates positions and weights, aligns returns, computes aggregate series and owns the public simulation/report methods.
- **RateSeries (optional specialised class):** can inherit from or compose the generic time-series model while carrying tenor/curve metadata.
- **OptionQuote / OptionSurfacePoint (advanced):** strike, expiry, side, bid/ask/mid, underlying price, IV and Greeks; never required by the core library.

6.2 Interface contracts

```
class MarketDataProvider(ABC):
    def get_price_series(self, symbol, start, end, frequency): ...
    def get_many(self, symbols, start, end, frequency): ...

class OptionDataProvider(MarketDataProvider):
    def get_option_chain(self, symbol, expiry=None, **filters): ...

class SimulationModel(ABC):
    def calibrate(self, data): ...
    def simulate(self, n_paths, horizon, seed=None): ...
```

7. Ingestion, cleaning and preprocessing

Data engineering deserves explicit treatment because the brief specifically calls out NaNs and different market calendars. A multi-market portfolio cannot safely be built by concatenating raw series and calling .dropna() without documenting the rule.

1. Retrieve raw data through a provider adapter and record source/timestamp metadata.
2. Normalize column names, time zone/date index, currency labels and numerical types.
3. Deduplicate timestamps and sort chronologically.
4. Apply corporate-action convention consistently (prefer adjusted closes for return analysis unless the test explicitly asks for raw prices).
5. Align assets on an agreed calendar/frequency; define whether missing observations are forward-filled, excluded, or treated as non-trading days.
6. Compute arithmetic or log returns only after alignment.
7. Validate minimum history, stale series, missing data and weight constraints.
8. Cache downloaded data where appropriate to avoid repeated API calls and rate-limit problems.

Input policy decision to document

The brief asks whether the tool should accept any time series or impose stricter restrictions. Recommended answer: keep the internal TimeSeries abstraction generic, but Portfolio should require a numerical monotonic DatetimeIndex, enough observations, known frequency/currency where relevant, and explicit weight rules. Fail fast with informative validation errors.

8. Analytics layer

Mean and standard deviation are compulsory. We can add a concise set of high-value statistics that directly support the simulation and the report without turning the project into a full performance-attribution system.

Metric	Core / extension	Purpose
Mean return and standard deviation	CORE	Explicit workshop requirement
Log / arithmetic returns	CORE	Input to portfolio and simulation calculations
Annualised return and volatility	CORE+	Readable investment interpretation
Covariance / correlation matrix	CORE+	Dependency structure for portfolio MC
Cumulative return / drawdown	CORE+	Portfolio-path diagnostics and plots
Skewness / excess kurtosis	EXTENSION	Motivation for Student-t vs Gaussian
Sharpe ratio	EXTENSION	Simple risk-adjusted performance diagnostic
VaR / Expected Shortfall	EXTENSION	Tail-risk comparison across simulation models
PCA on rates	ADVANCED	Possible level/slope/curvature fixed-income factor model

9. Monte Carlo engine

9.1 Baseline: correlated Gaussian Monte Carlo

The compulsory model should be understandable, configurable and correct. For each asset, historical data can provide an estimated drift and volatility (or the user can override them). Cross-asset covariance/correlation must be preserved so that the portfolio is not simulated as a set of independent assets.

Inputs:
- n_paths
- horizon (days or years)
- time_step
- expected return assumptions
- volatility assumptions
- correlation/covariance matrix
- random seed

Outputs:
- simulated asset paths
- simulated portfolio paths
- terminal portfolio-value distribution
- summary risk metrics and plots

For a lognormal equity-style process, one convenient discrete form is:

$$S(t+dt) = S(t) * \exp[(mu - 0.5*sigma^2)dt + sigma*sqrt(dt)*Z]$$

For a multi-asset portfolio, Z is a correlated vector obtained from the estimated correlation matrix (for example by Cholesky decomposition). Portfolio value is recomputed path by path, after applying the configured weights and any chosen rebalancing assumption.

9.2 Fat-tail extension: Student-t

Adding a Student-t alternative is strongly recommended because it is quantitatively meaningful and inexpensive to implement. Financial returns often exhibit excess kurtosis, so a Gaussian model can understate the probability of extreme outcomes.

If T follows a Student-t distribution with degrees of freedom nu > 2, its variance is nu/(nu-2). To compare a Student-t model with a Gaussian model at the same stated volatility, standardise the innovation using:

$$Z = T_nu * sqrt((nu - 2) / nu)$$

- **Calibration:** estimate degrees of freedom from historical returns where feasible instead of hard-coding a number.
- **Portfolio dependence:** use a multivariate Student-t construction rather than drawing each asset independently.
- **Comparison:** run Gaussian and Student-t simulations with the same horizon, volatility inputs and correlation assumptions.
- **Presentation value:** show how terminal tails, VaR and Expected Shortfall change when the innovation distribution is fat-tailed.

9.3 Fixed-income modelling: scope discipline

For the compulsory project, sovereign exposure can be represented through a bond ETF or price series, which keeps all assets compatible with the same portfolio machinery. A more advanced extension could model yield changes directly and revalue bonds using duration/convexity or PCA level/slope/curvature factors. The rates-driven version should only be attempted after the common price-series workflow is complete.

10. Optional options / implied-volatility extension

This module is a genuine differentiator, but it should be isolated from the compulsory workflow. The repository must still run end-to-end without an option-data API key.

10.1 Data retrieval

- **Primary candidate:** MarketData.app option-chain API, retrieving strike, expiration, side, bid/ask/last, underlying price, implied volatility and Greeks where available.
- **Independent cross-check:** yfinance option_chain() can provide a second source for option prices and implied volatility.
- **Optional adapters:** IBKR and Massive/Polygon can be implemented only if credentials, time and entitlements permit.

10.2 Own implied-volatility solver

Rather than trusting a vendor IV field blindly, we can calculate implied volatility ourselves from the market option price. Given S, K, T, r, q and an option price C_market, solve the Black-Scholes equation numerically for sigma. This is academically useful because it combines data ingestion, numerical methods and validation.

```
market option quote
  -> clean bid/ask and compute mid
  -> obtain S, K, T, r, q
  -> solve Black-Scholes price(sigma) = market price
  -> compare our IV with vendor IV
  -> build points {K, T, IV}
  -> interpolate / smooth an implied-volatility surface
```

10.3 Local-volatility idea - advanced only

If the core and Student-t extensions are fully stable, a final experiment could transform an implied-volatility surface into a local-volatility surface and use it in a risk-neutral Monte Carlo. This should be kept conceptually separate from the historical/physical Gaussian and Student-t portfolio models: the option-implied model answers a market-pricing question, not a simple expected-return forecast.

Scope gate

Do not make Dupire/local volatility a dependency of the submission. It is the first feature to drop if integration, testing, README or the 5-minute presentation are not already complete.

11. Package and repository structure

```
project-root/
|-- README.md
|-- pyproject.toml            # or requirements.txt
|-- .gitignore
|-- .env.example              # names only, never real secrets
|-- src/
|   `-- bme_toolkit/
|       |-- data/
|       |   |-- base.py
|       |   |-- yahoo.py
|       |   |-- alpha_vantage.py
|       |   |-- fred.py
|       |   |-- ecb.py
|       |   `-- marketdata_options.py     # optional
|       |-- domain/
|       |   |-- price_series.py
|       |   |-- portfolio.py
|       |   `-- option_data.py            # optional
|       |-- preprocessing/
|       |   `-- alignment.py
|       |-- analytics/
|       |   |-- statistics.py
|       |   `-- risk.py
|       |-- simulation/
|       |   |-- base.py
|       |   |-- gaussian.py
|       |   |-- student_t.py
|       |   `-- local_vol.py              # advanced
|       `-- reporting/
|           `-- plots.py
|-- tests/
|-- examples/
`-- docs/
    `-- architecture.png / uml.*
```

The public API should remain small. Most users should be able to construct a provider, retrieve series, build a Portfolio and call portfolio.report(), portfolio.plots_report() and portfolio.simulate(...) without knowing implementation details.

12. Reporting and visualisation

12.1 .report() in Markdown

- Portfolio composition and weights.
- Data range, data sources and missing-data diagnostics.
- Mean return, volatility and selected additional statistics.
- Correlation summary and concentration warnings.

- Monte Carlo configuration and terminal distribution statistics.
- VaR / Expected Shortfall when the extension is enabled.
- Warnings for insufficient history, high missingness, invalid weights or stale data.

12.2 .plots_report()

- Cumulative performance of assets and portfolio.
- Correlation heat map.
- Portfolio value / return evolution.
- Monte Carlo path fan chart (avoid plotting all paths if unreadable).
- Terminal portfolio-value histogram / density.
- Normal vs Student-t terminal distribution comparison.
- Optional: drawdown plot and option-implied-volatility smile/surface.

13. Testing, reproducibility and engineering standards

Because 50% of the mark is the GitHub component, the repository should demonstrate engineering discipline. A small number of good tests will add more credibility than another half-finished quantitative model.

Area	Recommended control
Unit tests	Returns/statistics, weight validation, calendar alignment, Gaussian/Student-t variance scaling
Provider tests	Mock HTTP responses so tests do not depend on external services
Integration smoke test	One small live-data example per free provider; skip gracefully if key absent
Secrets	Environment variables / .env.example; never commit API keys
Logging	Provider name, symbol, date range, request status and fallback path
Reproducibility	Random seed exposed in simulation config
Type/validation	Type hints + clear ValueError/custom exceptions for bad inputs
Caching	Optional local cache to reduce API usage and make demos reliable
CI (nice-to-have)	GitHub Actions running unit tests on push

14. Suggested three-person work split

The split should be by architectural responsibility rather than by provider or by individual feature in isolation. Every workstream should agree the shared interfaces first.

Workstream	Primary owner	Main responsibilities	Dependencies / hand-off
A. Data & infrastructure	Person 1	Repository skeleton; provider ABC; Yahoo/Alpha Vantage/FRED/ECB adapters;	Delivers standardised PriceSeries/RateSeries objects

		configuration; caching; API errors; batch retrieval	to B and C
B. Domain & analytics	Person 2	DataClasses; cleaning/alignment; returns; stats; correlation; Portfolio; validation; report metrics	Consumes A interfaces; exposes clean portfolio state to C
C. Simulation & reporting	Person 3	Gaussian MC; Student-t extension; path aggregation; VaR/ES; plots_report; optional option/IV module	Consumes B Portfolio API; returns simulation-result objects used by reporting
Shared	All	README, UML, tests, integration demo, 5-minute presentation, code review	No feature is considered complete until another team member can run it

First team decision

Before anyone codes deeply, agree the signatures and schemas for MarketDataProvider, PriceSeries, Position, Portfolio and SimulationResult. This prevents three parallel implementations that later need painful integration.

15. Milestones to 12 November

Target date	Milestone	Exit criterion
6-12 Oct	Design and repository skeleton	Interfaces agreed; /src layout; dependency file; baseline README; first unit-test scaffolding
13-20 Oct	Core data layer	At least Yahoo + Alpha Vantage + one rates/macro provider working; standardised outputs; batch retrieval
21-28 Oct	Domain + analytics + Gaussian MC	PriceSeries/Portfolio complete; cleaning/alignment; correlated portfolio MC; basic plots
29 Oct-4 Nov	Differentiating layer	Student-t MC; risk comparison; .report(); .plots_report(); demonstration portfolio
5-8 Nov	Advanced option extension (only if green)	MarketData.app/yfinance option chain + IV solver; otherwise spend time on tests/docs
9-10 Nov	Integration and QA	Fresh clone works; secrets handled; tests pass; UML; README examples reproducible
11 Nov	Presentation rehearsal	5-minute demo timed; backup screenshots/results; final repository cleanup
12 Nov before 18:00	Submission	GitHub link delivered through virtual campus; tagged final commit/release

16. Risks and scope control

Risk	Impact	Mitigation

API instability / rate limits	Demo fails or returns incomplete data	Adapters + caching + fixtures; no single provider dependency
Over-scoping local vol/options	Core deliverable remains unfinished	Hard gate: advanced work starts only after core tests/docs pass
Calendar misalignment	Incorrect covariance/portfolio returns	Central alignment policy with explicit tests
Mixing physical and risk-neutral models	Incorrect interpretation of option-implied simulations	Keep historical Gaussian/Student-t and Q-measure option models in separate classes/documentation
Hard-coded API secrets	Security/repository issue	.env.example + environment variables
Inconsistent team code	Integration delay	Shared interfaces, code review and small pull requests
Five-minute presentation too dense	Good code but weak grade on 50% presentation	Show one end-to-end story and one quantitative differentiator; avoid feature catalogue

17. Definition of done and presentation storyline

17.1 Core definition of done

- Fresh clone installs and runs with documented commands.
- At least two equity/index data providers and one additional data category are integrated.
- Provider outputs are standardised into common DataClasses.
- Batch extraction works for multiple assets.
- Portfolio construction and weight validation work.
- Mean/std are automatic and additional analytics are available.
- Correlated Gaussian Monte Carlo runs for one asset and the full portfolio.
- .report() and .plots_report() are implemented.
- NaNs/calendar alignment have explicit, tested rules.
- UML/architecture diagram and README are complete.

17.2 Differentiation definition of done

- Student-t model is variance-standardised and compared against Gaussian MC.
- Degrees of freedom can be user-specified or estimated from data.
- VaR/Expected Shortfall or equivalent tail-risk measures demonstrate the practical effect of fat tails.
- Optional option-data module retrieves a filtered chain and can solve implied volatility independently.

17.3 Five-minute presentation storyline

1. Problem (30 sec): multiple providers create inconsistent data and fragile scripts.

2. Architecture (60 sec): show provider abstraction -> standardisation -> Portfolio -> Simulation -> Reports.

3. Live/demo workflow (90 sec): retrieve several assets, build the 70/30 portfolio, show analytics.

4. Quant differentiator (90 sec): Gaussian vs Student-t terminal distribution and tail risk.

5. Engineering close (30 sec): plug-and-play GitHub, tests, ability to add another provider/model without changing the core.

Appendix A. Suggested interfaces and equations

A.1 SimulationResult object

```
@dataclass
class SimulationResult:
    model_name: str
    paths: np.ndarray
    terminal_values: np.ndarray
    horizon: float
    n_paths: int
    seed: int | None
    diagnostics: dict

    def var(self, level=0.99): ...
    def expected_shortfall(self, level=0.99): ...
```

A.2 Portfolio aggregation

For aligned asset returns r_t and weight vector w, the simple static-weight portfolio return is r_p,t = w^T r_t. The covariance-based portfolio variance is w^T Sigma w. These quantities provide calibration inputs and validation checks for the Monte Carlo engine.

A.3 Student-t multivariate construction

One practical construction uses correlated Gaussian draws divided by the square root of a common chi-square factor. This creates joint tail dependence while preserving a chosen correlation structure. The implementation must be carefully rescaled so the configured volatility remains interpretable.

A.4 Implied-volatility inversion

For the advanced option module, solve BS_price(S, K, T, r, q, sigma) = market_price for sigma using a robust root finder such as Brent. Reject quotes violating basic no-arbitrage bounds or having unusable bid/ask spreads before calibration.

Appendix B. Source notes

Workshop requirements are derived from the uploaded BME Institute document "Taller: B1-T1 - Toolkit de Extraccion, Analisis Bursatil y Gestion de Carteras". The provider notes below were checked against current official documentation on 6 October 2026; commercial plans and entitlements may change and should be re-checked during implementation.

BME Institute workshop brief: Uploaded file: Taller_B1_T1.pdf

yfinance documentation: https://ranaroussi.github.io/yfinance/

Alpha Vantage API documentation: https://www.alphavantage.co/documentation/

FRED API documentation: https://fred.stlouisfed.org/docs/api/fred/

ECB Data Portal API / SDMX: https://data.ecb.europa.eu/help/api/data

MarketData.app options API: https://www.marketdata.app/docs/api/options/

MarketData.app free-account limits: https://www.marketdata.app/docs/account/free-accounts/

Interactive Brokers delayed-market-data docs: https://www.interactivebrokers.com/docs/tws-api/doc/market-data-delayed/introduction

Interactive Brokers market-data pricing / minimum equity: https://www.interactivebrokers.com/en/pricing/market-data-pricing.php

Team recommendation

Treat the assignment as a clean software-engineering project with a credible quant use case. Make the common interfaces, standardisation, portfolio model and baseline Monte Carlo impeccable. Add Student-t as the primary quantitative differentiator. Treat option-implied volatility and local volatility as plug-in extensions that showcase extensibility rather than as dependencies of the submission.